



HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION
TECHNOLOGY

PROJECT REPORT

IPSP: An Image Processing and Storage Pipeline for Traffic
Analysis

Course: Big Data Storage and Processing (IT4043E)

Supervisor: Dr. Tran Viet Trung

Authors:

Vu Huu An	20225467
Luu Thien Viet Cong	20225477
Dam Quang Duc	20225483
Lai Tri Dung	20225486
Nguyen Trong Tam	20225527

Hanoi, December 2025

Mục lục

1	Problem Definition	2
1.1	Project Introduction	2
1.2	Data Introduction	3
1.3	Problem Suitability for Big Data	3
1.4	Scope and Limitations	3
2	Architecture and Design	4
2.1	Overall Architecture	4
2.1.1	Lambda Architecture System	4
2.1.2	Justification for Lambda Architecture	4
2.2	Detailed components and their roles	4
2.2.1	Data Ingestion and Event Buffering Layer	5
2.2.2	Dual-Path Processing Engine	5
2.2.3	Polyglot Persistence Layer	5
2.3	Data Flow and Component Interaction Diagrams	6
2.3.1	End-to-End Data Pipeline	6
2.3.2	Kubernetes Orchestration and Service Integration	6
3	Implementation Details	7
3.1	Source code with full documentation	7
3.2	Environment-specific configuration files	8
3.2.1	Data Lake Storage (MinIO)	8
3.2.2	Message Broker (Kafka)	9
3.2.3	Spark Speed Layer (Spark Stream)	9
3.2.4	Spark Batch Layer - Model Training	10
3.2.5	Spark Batch Layer - Metrics Computing	10
3.2.6	Serving & Visualization Layer	11
3.3	Deployment Strategy & Workload Orchestration	12
3.3.1	Data Ingestion (Producer)	12
3.3.2	Streaming Processor (Spark Stream)	13
3.3.3	Batch Processing (Spark Batch Model Training)	13
3.3.4	Batch Processing (Spark Batch Metric Computing)	15
4	Lessons Learned	16
4.1	Lessons on Data Ingestion: Solving the "Small Files Problem" with LMDB	16
4.1.1	Problem Description	16
4.1.2	Approaches Tried	16
4.1.3	Final Solution	17
4.1.4	Key Takeaways	17
4.2	Lessons on Data Processing with Spark: The "Distributed Overhead" Paradox	17
4.2.1	Problem Description	17
4.2.2	Approaches Tried	17
4.2.3	Final Solution	17
4.2.4	Key Takeaways	17
4.3	Lessons on Stream Processing: Handling Late Data with Watermarking	18
4.3.1	Problem Description	18
4.3.2	Approaches Tried	18
4.3.3	Final Solution	18
4.3.4	Key Takeaways	18
4.4	Lessons on Data Storage: Hybrid Storage Strategy (Hot vs. Cold)	18
4.4.1	Problem Description	18
4.4.2	Approaches Tried	18
4.4.3	Final Solution	18

4.4.4	Key Takeaways	18
4.5	Lessons on System Integration: Service Discovery in Kubernetes	19
4.5.1	Problem Description	19
4.5.2	Approaches Tried	19
4.5.3	Final Solution	19
4.5.4	Key Takeaways	19
4.6	Lessons on Performance Optimization: Decoupling Training Workloads for GPU Efficiency	19
4.6.1	Problem Description	19
4.6.2	Approaches Tried	19
4.6.3	Final Solution	19
4.6.4	Key Takeaways	20
4.7	Lessons on Monitoring & Debugging: Debugging Distributed UDF Failures	20
4.7.1	Problem Description	20
4.7.2	Approaches Tried	20
4.7.3	Final Solution	20
4.7.4	Key Takeaways	20
4.8	Lessons on Scaling: Event-Driven Horizontal Scaling Strategies	20
4.8.1	Problem Description	20
4.8.2	Approaches Tried	20
4.8.3	Final Solution	21
4.8.4	Key Takeaways	21
4.9	Lessons on Data Quality & Testing: Schema Enforcement in Streaming	21
4.9.1	Problem Description	21
4.9.2	Approaches Tried	21
4.9.3	Final Solution	21
4.9.4	Key Takeaways	21
4.10	Lessons on Security & Governance: Secret Management Trade-offs	21
4.10.1	Problem Description	21
4.10.2	Approaches Tried	21
4.10.3	Final Solution	22
4.10.4	Key Takeaways	22
4.11	Lessons on Fault Tolerance: Ensuring Exactly-Once Semantics with Checkpointing	22
4.11.1	Problem Description	22
4.11.2	Approaches Tried	22
4.11.3	Final Solution	22
4.11.4	Key Takeaways	22
5	Conclusion and Future Work	22
5.1	Conclusion	22
5.2	Future Work	23

1 Problem Definition

1.1 Project Introduction

The rapid acceleration of urbanization has rendered traffic congestion a critical challenge in modern metropolitan areas. Persistent gridlocks during peak hours not only impede daily mobility but also result in significant economic losses and environmental degradation. To mitigate these issues, Intelligent Transportation Systems (ITS) require the capability to provide timely, granular information regarding traffic density across specific road segments. Such data empowers commuters to select optimal routes, thereby distributing traffic flow and reducing overall travel time.

In this context, the proposed project addresses the problem of real-time traffic analysis utilizing computer vision techniques on unstructured image data. The primary objective is to extract meaningful quantitative

indicators—specifically vehicle flow rate, traffic density, and vehicle classification—to facilitate data-driven decision-making for urban traffic management.

1.2 Data Introduction

The experimental framework utilizes a comprehensive dataset comprising 11,508 traffic images, sourced from a competitive benchmark. The dataset is accessible for replication purposes via: <https://drive.google.com/drive/u/1/folders/1z1LvF1dJn7C29yZDsCrd0s1c1VcYLSiC>.

These images capture a diverse range of urban traffic scenarios, featuring heterogeneous vehicle types including motorbike, cars, buses, and trucks. The data encompasses 10 camera angles. Crucially, each image is accompanied by comprehensive ground truth annotations, which specify the spatial coordinates (bounding boxes) and corresponding class identifiers for all vehicles present within the frame. The volume and sequential nature of this dataset allow it to effectively simulate high-velocity real-time camera feeds, rendering it highly suitable for streaming processing pipelines.

1.3 Problem Suitability for Big Data

The characteristics of the proposed traffic analysis system inherently align with the fundamental "Vs" of the Big Data paradigm, necessitating the use of distributed computing frameworks over traditional monolithic architectures:

- **Volume:** While the initial dataset contains 11,508 images, a production-grade deployment involving continuous feeds from city-wide surveillance networks would generate petabytes of video data. Traditional storage and processing systems cannot scale linearly to accommodate this magnitude of unstructured data.
- **Velocity:** The system simulates high-speed data ingestion akin to live video streams. To be actionable, traffic insights must be generated in near real-time, requiring low-latency processing pipelines capable of handling high throughput without creating backlogs.
- **Variety:** The dataset exhibits high heterogeneity, combining unstructured raw imagery with structured ground truth annotations (spatial coordinates/bounding boxes, class IDs) and metadata. This requires the system to process and correlate multi-modal data formats—linking pixel data with numerical vectors and semi-structured logs (e.g., JSON-formatted insights)—necessitating flexible schema management.
- **Veracity:** Real-world sensor data is inherently noisy. Factors such as occlusion, varying illumination, weather conditions, and camera jitter introduce uncertainty. Robust preprocessing and fault-tolerant AI inference models are required to ensure data quality and reliability.

Given these constraints, traditional I/O-bound systems would inevitably face bottlenecks. Consequently, Big Data technologies such as Apache Spark (for distributed processing) and Apache Kafka (for message buffering) are essential to ensure scalability, fault tolerance, and high availability.

1.4 Scope and Limitations

Scope: The project focuses on the end-to-end implementation of a Lambda-like architecture designed for urban traffic monitoring. The pipeline encompasses data ingestion, distributed preprocessing, deep learning-based inference, object storage, and visualization of derived metrics.

Limitations:

- **Data Constraints:** The use of a static, educational dataset may limit the model's generalization capabilities compared to a production system trained on continuous, multi-season data.
- **Live Integration:** The current implementation simulates streaming via file replay rather than integrating directly with live IP camera protocols (RTSP/RTMP).

- **Hardware Dependency:** Inference is assumed to be CPU-bound for demonstration purposes. In a high-load production environment, GPU acceleration would be mandatory to meet strict latency SLAs.
- **Storage Latency:** Potential input/output latency may occur during fetch operations from object storage (MinIO) under extreme concurrency, necessitating optimization strategies such as caching or edge computing in future iterations.

2 Architecture and Design

This section details the architectural decisions made for the project. It justifies the selection of the Lambda Architecture and provides an in-depth analysis of the data flow and component roles within the Kubernetes ecosystem.

2.1 Overall Architecture

2.1.1 Lambda Architecture System

Initially designed as a pure Kappa architecture, our system has evolved into Lambda architecture to balance the trade-offs between real-time responsiveness and deep historical analytics. This model bifurcates the data processing responsibility into two distinct but complementary layers:

- **Speed Layer (Stream Processing):** Leverages **Apache Kafka** for high-throughput data ingestion and **Apache Spark Structured Streaming** for real-time processing. This layer handles the immediate consumption of video frames, performing transient analysis before persisting the raw data into **MinIO** storage in **LMDB** format to ensure data durability for downstream tasks.
- **Batch Layer (Historical Analytics):** Also powered by **Apache Spark**, this layer operates on the data lake (MinIO) and is architected into two distinct, parallel execution pipelines:
 1. *Model Retraining Pipeline:* Ingests historical annotated datasets (images and labels stored as LMDB) from MinIO to iteratively retrain and optimize the object detection models. The updated model artifacts are subsequently versioned and stored back into MinIO.
 2. *Metric Computation Pipeline:* Processes historical data from MinIO to derive complex traffic indicators, such as vehicle density and average speed estimation (utilizing homography techniques). The resulting structured insights are serialized in JSON format and archived in **MongoDB** for efficient querying and reporting.

2.1.2 Justification for Lambda Architecture

The transition to a hybrid approach is necessitated by three fundamental technical requirements:

1. **Comprehensive Insight Generation:** While the Speed Layer provides immediate "per-second" metrics, the Batch Layer is essential for calculating complex temporal aggregates.
2. **Algorithm Iteration and Backfilling:** The Batch Layer serves as a "ground truth" environment. When a new Computer Vision UDF is developed, the system can "backfill" historical data stored in MinIO to update past analytics without disrupting the real-time pipeline.
3. **Infrastructure Synergy on Kubernetes:** By utilizing a shared container orchestration layer, the Batch and Stream jobs can share the same executor images, ensuring environment parity while dynamically adjusting resource quotas based on cluster load.

2.2 Detailed components and their roles

The system architecture embodies a robust implementation of the **Lambda Architecture**, designed to balance low-latency stream processing with high-precision batch analytics. Orchestrated within a **Kubernetes (K8s)** environment, the stack ensures high availability, fault tolerance, and horizontal scalability across all layers.

2.2.1 Data Ingestion and Event Buffering Layer

This layer acts as the entry point for massive streams of high-resolution video frame data, ensuring decoupling between data producers (cameras/sensors) and consumers (processing engines).

- **Apache Kafka (Distributed Event Streaming):** Serves as the high-throughput backbone for the system. It buffers ingress traffic to prevent back-pressure on the ingestion nodes during traffic spikes.
- **Partitioning Strategy:** To preserve the causal ordering of events—a critical requirement for velocity estimation—the `traffic_data` topic is configured with a strategic partitioning key based on the `camera_id`. This semantic partitioning ensures that all frames from a single camera are routed to the same partition and consumer instance, guaranteeing strict chronological processing.

2.2.2 Dual-Path Processing Engine

The core processing logic is bifurcated into two parallel pathways, optimizing for either latency or accuracy.

- **Stream Processing Unit (Speed Layer):** Built on lightweight **Python workers** and **Redis**, this layer prioritizes sub-second latency over absolute precision.
 - *Mechanism:* It performs real-time object detection and greedy-matching tracking. State management for speed calculation is handled via ephemeral Redis keys (30-second TTL), allowing for immediate "Jam/Crowded" alert generation based on pixel density thresholds.
 - *Role:* Delivers instantaneous traffic status updates and operational alerts.
- **Batch Analytics Engine (Batch Layer):** Powered by **Apache Spark (PySpark)** and **Ultralytics YOLO**, this layer focuses on deep learning inference and precision metrics.
 - *Mechanism:* It executes periodic, high-compute jobs that re-process historical data using high-fidelity Deep Learning models (YOLOv8). Unlike the approximation methods in the Speed Layer, this engine utilizes **Homography Matrix** transformations to map 2D pixel coordinates to real-world metric coordinates, ensuring high accuracy for velocity and distance metrics.
 - *Role:* Generates historical statistics (1-minute and daily aggregations), traffic heatmaps, and re-trains models for continuous improvement.

2.2.3 Polyglot Persistence Layer

The architecture employs a **Polyglot Persistence** strategy, utilizing best-of-breed storage solutions tailored to specific data structures and access patterns.

- **MinIO (Object Storage):** Functions as the centralized Data Lake and S3-compatible storage layer.
 - *Artifacts:* Stores unstructured data such as raw video frames, generated heatmap images (PNG), trained model weights (`.pt` files), and static JSON configuration files defining Road Geometry and Region of Interest (ROI) masks.
- **MongoDB (Document Store):** Acts as the high-performance Serving Layer for the application frontend.
 - *Artifacts:* Stores structured, hierarchical analytical results, including 1-minute aggregated traffic stats, daily summaries, and metadata links to heatmaps. Its flexible schema supports the rapid iteration of the analytical data model.
- **Redis (In-Memory Data Structure Store):** Provides transient state storage for the Stream Processing Unit.
 - *Artifacts:* Caches the "previous frame" state (object centroids and timestamps) required for real-time velocity deltas and acts as a deduplication buffer for alert notifications.

2.3 Data Flow and Component Interaction Diagrams

The architectural design, as depicted in Figure 1, implements a split-merge pipeline that segregates the high-velocity ingestion path from the high-latency analytical path, unifying them only at the persistence layer.

2.3.1 End-to-End Data Pipeline

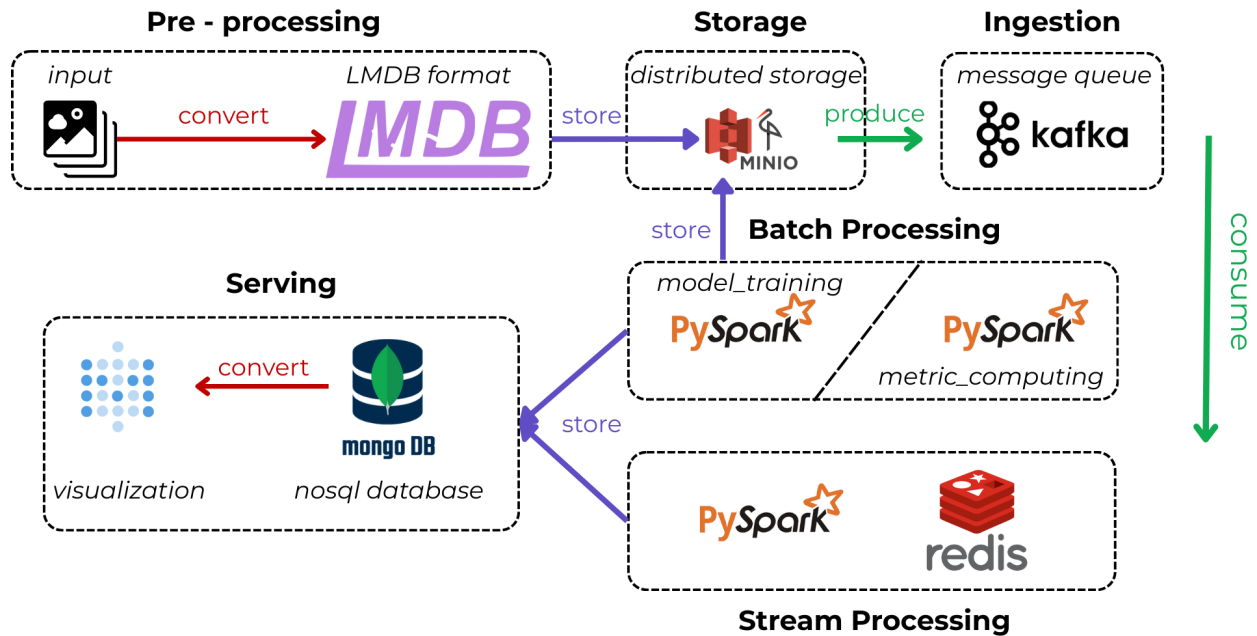
The data lifecycle follows a four-stage process designed to handle high-resolution visual data efficiently:

1. **Pre-processing and Storage-First Ingestion:** To optimize throughput and avoid overburdening the message broker with large binary payloads, the system adopts a *Claim-Check* pattern.
 - Raw input images are first serialized into the **LMDB (Lightning Memory-Mapped Database)** format and persisted immediately to **MinIO** distributed storage.
 - Upon successful storage, an event notification (containing metadata and storage pointers) is produced to **Apache Kafka**. This ensures that the message queue remains lightweight while providing a persistent source of truth in the Data Lake.
2. **Dual-Path Consumption (Lambda Architecture):** The Kafka consumers bifurcate the data flow into two parallel processing streams:
 - **Stream Processing Layer:** Dedicated Python consumers subscribe to the Kafka feed for real-time analysis. This layer utilizes **Redis** for ephemeral state management (e.g., tracking object centroids between frames for immediate speed estimation) to minimize latency.
 - **Batch Processing Layer:** Powered by **PySpark**, this layer consumes historical data windows. It performs heavy-lifting tasks such as *model_training* (retraining YOLO models) and *metric_computing* (calculating precise homography-based velocity and generating heatmaps).
3. **Unified Serving Layer:** Despite the divergent processing paths, all analytical results converge in the Serving Layer. **MongoDB** acts as the consolidation point, storing real-time alerts from the Stream Layer alongside aggregated statistical metrics from the Batch Layer.
4. **Visualization:** The presentation layer queries the structured documents in MongoDB to render real-time traffic dashboards and historical heatmaps for end-users.

2.3.2 Kubernetes Orchestration and Service Integration

The physical deployment is abstracted via Kubernetes (K8s) to ensure resilience and service discovery:

- **Stateful Component Orchestration:** Given the data-intensive nature of the pipeline, critical data stores (Kafka, MongoDB, MinIO, Redis) are orchestrated as Kubernetes **StatefulSets**. Unlike stateless deployments, this configuration assigns a sticky identity (e.g., `kafka-0`, `minio-0`) and stable network ID to each pod. Crucially, it ensures automated binding to **PersistentVolumeClaims** (PVCs). This guarantees that the heavy image data buffered in Kafka and the historical analytics in MongoDB survive pod restarts or node rescheduling, maintaining data integrity for both the Stream and Batch consumers.
- **Internal Service Discovery & Interaction Flow:** The interaction between decoupled components is mediated by Kubernetes CoreDNS, abstracting the dynamic IP addressing of pods:
 - **Ingestion Path:** The Producer service resolves the `my-kafka` service at port 9092 to push base64-encoded frames directly into the message bus.
 - **Resource Fetching:** Upon initialization, the Spark Batch Job connects to `my-minio:9000` via HTTP to download the YOLO model weights (`.pt`) and road geometry configurations (`.json`) before commencing inference.



Hình 1: System Architectural Diagram: Unified Hybrid Ingestion and Processing Pipeline.

- **State Synchronization:** The Stream Processor maintains a low-latency connection to `redis-service:6379` to fetch and update vehicle tracking states for velocity calculation.

This DNS-based discovery eliminates hard-coded dependencies, allowing the computation layer (Spark) and storage layer (Kafka/MinIO) to scale independently without reconfiguration.

3 Implementation Details

3.1 Source code with full documentation

The project source code is organized into a modular architecture, strictly separating Infrastructure-as-Code (IaC) from application logic. The repository structure mirrors the Lambda Architecture components, comprising four primary modules: Orchestration, Ingestion, Speed Layer, and Batch Layer.

The full implementation is available at:

<https://github.com/laitridung2004/IPSP-An-image-processing-and-storage-pipeline>

```

1 /project-root
2 |-- /1_kubernetes                                # Deployment & Orchestration Layer
3 |   |-- 00-minio.yaml                            # Data Lake & Object Storage (MinIO)
4 |   |-- 01-kafka.yaml                            # Distributed Event Streaming (Kafka)
5 |   |-- 02-mongodb.yaml                          # Metric Serving Layer (NoSQL)
6 |   |-- 03-spark-batch-cluster.yaml              # Batch Processing Cluster Base
7 |   |-- 03-spark-batch-computing.yaml            # Batch Computing Logic Specs
8 |   |-- 03-spark-batch-submit-job.yaml           # Batch Job Submission Configuration
9 |   |-- 03-spark-batch-training.yaml              # Model Training Resource Specs
10 |  |-- 03-spark-stream-cluster.yaml              # Stream Processing Cluster Base
11 |  |-- 03-spark-stream-submit-job.yaml           # Stream Job Submission Configuration
12 |  |-- 04-producer.yaml                          # Ingestion Producer Deployment
13 |  |-- 05-metabase.yaml                         # BI & Visualization Dashboard
14 |  |-- 06-redis.yaml                             # Fast-access State Store (In-memory)

```



```

15 |
16 | -- /2_ingestion                # Data Sourcing Logic
17 | | -- Dockerfile                # Container environment definition
18 | | -- producer.py               # Multi-threaded LMDB-to-Kafka ingestion
19 |
20 | -- /3_stream_processing         # Speed Layer (Near Real-time)
21 | | -- Dockerfile                # Spark Stream container definition
22 | | -- main.py                   # Spark Structured Streaming pipeline
23 | | -- ttl.py                    # Time-to-live / Retention logic
24 | | -- util.py                   # Redis integration for stateful lookups
25 |
26 | -- /4_batch_processing          # Batch Layer (Offline Analytics)
27 | | -- 1_model_training/         # Spatial & Perspective analytics
28 | | | -- dispatcher/
29 | | | | -- dispatcher_main.py    # Entry point for dispatcher
30 | | | | -- job_dispatcher.py     # Job distribution logic
31 | | | -- gpu_worker/
32 | | | | -- job_worker.py         # Worker processing logic
33 | | | | -- run_worker.bat        # Execution script (Windows)
34 | | | | -- run_worker.sh         # Execution script (Linux/Unix)
35 | | | | -- worker_main.py        # Entry point for worker
36 | | | -- Dockerfile              # Training environment definition
37 | | | -- config.py               # Configuration parameters
38 | | | -- data_processor.py        # Data preprocessing modules
39 | | | -- minio_uploader.py        # Artifact upload to Data Lake
40 | | | -- model_trainer.py         # Core model training loop
41 | | | -- util.py                  # Utility helper functions
42 | | -- 2_metric_computing/        # MLOps: Automated YOLO re-training
43 | | | -- Dockerfile              # Metric computing environment
44 | | | -- main.py                  # Metric calculation pipeline
45 | | | -- util.py                  # Metric utility functions

```

Listing 1: Project Directory Structure and Component Mapping

3.2 Environment-specific configuration files

To ensure reproducibility and scalability, the system infrastructure is defined through declarative Kubernetes manifests. Below is a detailed analysis of the critical configurations for each core service.

3.2.1 Data Lake Storage (MinIO)

MinIO serves as the centralized object storage for raw images and model artifacts. It is deployed as a **StatefulSet** to guarantee data persistence.

```

1 # From: 00-minio.yaml
2 kind: StatefulSet
3 spec:
4   replicas: 1
5   volumeClaimTemplates:
6   - metadata:
7     name: minio-data
8     spec:
9       accessModes: [ "ReadWriteOnce" ]
10      resources:
11        requests:
12          storage: 5Gi

```

Listing 2: MinIO Persistence Configuration

Key Configuration:

- **StatefulSet & VolumeClaimTemplates:** Unlike standard Deployments, this ensures that MinIO binds to a stable Persistent Volume (PV). If the Pod restarts, the data stored in `/data` (buckets, images, models) remains intact.
- **Storage Request:** We allocate 5Gi as the baseline for the Data Lake, which is sufficient for the traffic dataset and model versions in this POC phase.

3.2.2 Message Broker (Kafka)

Kafka acts as the high-throughput backbone for the streaming pipeline. The configuration focuses on partition strategy and network accessibility.

```
1 # From: 01-kafka.yaml
2 env:
3   - name: KAFKA_NUM_PARTITIONS
4     value: "10"
5   - name: KAFKA_ADVERTISED_LISTENERS
6     value: "INTERNAL://...:9092,EXTERNAL://...:19092"
7   - name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR
8     value: "1"
```

Listing 3: Kafka Partitioning & Networking

Key Configuration:

- **KAFKA_NUM_PARTITIONS = 10:** This is a critical tuning parameter. By creating 10 partitions, we allow up to 10 Spark executors to consume data in parallel, maximizing throughput.
- **Dual Listeners:** We configure separate listeners for **INTERNAL** (Kubernetes internal network, port 9092) and **EXTERNAL** (Host machine access, port 19092), facilitating both internal service communication and external debugging.

3.2.3 Spark Speed Layer (Spark Stream)

The Streaming Cluster is optimized for low-latency processing and high concurrency.

```
1 # From: 03-spark-stream-cluster.yaml
2 kind: Deployment
3 metadata:
4   name: spark-worker
5 spec:
6   replicas: 2
7   template:
8     spec:
9       containers:
10        - resources:
11            requests: {memory: "1Gi", cpu: "1000m"}
12            limits: {memory: "2Gi", cpu: "2000m"}
13 # From: 03-spark-stream-submit-job.yaml
14 env:
15   - name: SPARK_DRIVER_POD_IP
16     valueFrom: {fieldRef: {fieldPath: status.podIP}}
17 command:
18   - "--conf"
19   - "spark.executor.memoryOverhead=512m"
20   - "spark.driver.host=$(SPARK_DRIVER_POD_IP)"
```

Listing 4: Stream Cluster & Job Config

Key Configuration:

- **Worker Replicas (2):** We deploy 2 worker pods, providing physical resources for parallel processing.
- **Memory Overhead (512m):** The streaming job requires a higher overhead (512MB) compared to batch jobs. This buffer is essential for the Kafka Consumer buffer.
- **Client Mode Networking:** We dynamically inject `SPARK_DRIVER_POD_IP` to allow Executors to communicate back to the Driver running in the submit-pod.

3.2.4 Spark Batch Layer - Model Training

This module is isolated from the standard Spark cluster. It runs as a standalone deployment dedicated to the heavy workload of retraining the YOLO model.

```
1 # From: 03-spark-batch-training.yaml
2 kind: Deployment
3 metadata:
4   name: traffic-yolo-train
5 spec:
6   containers:
7   - name: yolo-dispatcher
8     image: anvui204/yolo-training-batch:v9
9     resources:
10      requests:
11        memory: "4Gi"
12        cpu: "1"
13      limits:
14        memory: "8Gi"
15        cpu: "2"
16     env:
17     - name: SPARK_MASTER
18       value: "local[*]"
```

Listing 5: Training Resource Isolation

Key Configuration:

- **Resource Guarantee (4Gi/1CPU):** Training is the most resource-intensive task. We define high requests to ensure the Scheduler only places this Pod on a node with sufficient capacity.
- **Local Spark Master:** Unlike other services, this sets `SPARK_MASTER=local[*]`. It uses Spark purely as a library for local data processing within the container before triggering the PyTorch/YOLO training loop, avoiding network overhead with the cluster.

3.2.5 Spark Batch Layer - Metrics Computing

The Batch Computing cluster handles scheduled inference jobs. Unlike the streaming layer which optimizes for concurrency, this layer features a unique **Model Pre-fetching** strategy and precise resource isolation for heavy inference tasks.

```
1 # From: 03-spark-batch-cluster.yaml
2 initContainers:
3 - name: init-download-model
4   image: quaquucc/spark-batch-computing:v1
5   command: ["/bin/sh", "-lc"]
6   args:
7   - "python3 -c '...c.fget_object(..., '/tmp/yolov8_traffic_best.pt')'"
8 volumeMounts:
9 - mountPath: /tmp
```

```

10     name: temp-data
11
12 # From: 03-spark-batch-submit-job.yaml
13 env:
14   - name: SPARK_DRIVER_POD_IP
15     valueFrom: {fieldRef: {fieldPath: status.podIP}}
16 command:
17   - "--conf"
18   - "spark.executor.memoryOverhead=256m"
19   - "spark.executor.instances=1"

```

Listing 6: Batch Cluster & Inference Job Config

Key Configuration:

- **Model Pre-fetching (InitContainer):** We utilize an `initContainer` to download the YOLOv8 model weights (60MB) from MinIO to a shared volume (`/tmp`) before the Spark Worker starts. This eliminates network latency during the actual inference execution.
- **Resource Optimization (Overhead 256m):** The memory overhead is tuned to 256m (lower than the Stream layer's 512m) because batch jobs do not require maintaining continuous state buffers, freeing up more RAM for the actual inference computation.
- **Single-Instance Execution:** Configured with `replicas: 1` and `executor.instances=1`, concentrating resources on a single, powerful node to maximize the performance of the monolithic deep learning model.

3.2.6 Serving & Visualization Layer

This layer provides the interface for data consumption, consisting of MongoDB (Storage), Metabase (BI), and Redis (Caching).

```

1 # From: 02-mongodb.yaml
2 volumeClaimTemplates:
3   - metadata:
4     name: mongo-data
5     spec:
6       resources:
7         requests:
8           storage: 5Gi
9 # From: 05-metabase.yaml
10 kind: PersistentVolumeClaim
11 metadata:
12   name: metabase-pvc
13 spec:
14   resources:
15     requests:
16       storage: 2Gi

```

Listing 7: Persistence for Visualization

Key Configuration:

- **Metabase PVC:** We mount a 2Gi volume to `/metabase-data` to store the H2 database file (`metabase.db`). This ensures that dashboards, questions, and chart configurations are saved permanently.
- **MongoDB StatefulSet:** Similar to MinIO, MongoDB uses a StatefulSet to maintain stable network identity and persistent storage for the processed traffic metadata.

3.3 Deployment Strategy & Workload Orchestration

3.3.1 Data Ingestion (Producer)

To ensure the system’s reliability and performance, we adopted a segregated deployment strategy. The workloads are categorized into Data Ingestion (Producer) and Distributed Processing (Spark Clusters), each with a tailored orchestration logic.

The Producer serves as the entry point of the pipeline, simulating a real-world camera network. Unlike standard microservices, its deployment strategy focuses on **finite execution**, **concurrency simulation**, and **strict ordering**.

Since the producer simulates a specific dataset (traffic video), we deploy it as a Kubernetes **Job** with `restartPolicy: Never`.

- **Lifecycle:** The Pod starts, streams the dataset once, and terminates. This prevents resource wastage compared to a long-running Deployment.
- **Reproducibility:** The `CAM_MODE` environment variable allows us to switch between "Dev Mode" (10 cameras) and "Demo Mode" (different set of 10 cameras) without changing the code.

To simulate 10 high-frequency cameras (60 FPS) from a single Pod, we utilize a **Multi-threaded architecture**.

```
1 # From: producer.py
2 threads = []
3 for cam_id, path in local_paths.items():
4     t = threading.Thread(target=stream_single_camera, args=(cam_id, ...))
5     t.start()
```

Listing 8: Multi-threaded Camera Simulation Logic

Since pushing data to Kafka is an I/O-bound operation, Python threads release the Global Interpreter Lock (GIL) during network calls. This allows a single producer instance to saturate the network bandwidth effectively without CPU blocking.

Standard Kafka producers use Round-Robin partitioning, which is unsuitable for video streams as it scatters frames across brokers. We implemented a **Deterministic Partitioning Strategy** to guarantee temporal ordering.

$$P_{target} = (ID_{camera} - 1) \pmod{N_{partitions}}$$

```
1 target_partition = (cam_num - 1) % NUM_PARTITIONS
2 producer.send(..., partition=target_partition)
```

Listing 9: Ordering Guarantee Implementation

Strategic Benefit: This ensures that all frames from a specific camera (e.g., `cam_01`) are always routed to the same partition. Consequently, the Spark Structured Streaming engine receives frames in the exact sequence of generation, which is critical for time-series analysis.

To eliminate storage latency during the live stream, the producer executes a **Pre-fetching phase** during initialization.

- **Mechanism:** The application connects to MinIO, downloads the heavy LMDB data files to the Pod’s ephemeral storage (`/tmp`), and only starts streaming once the data is local.
- **Impact:** This decouples the streaming throughput from the MinIO read latency, ensuring a stable, jitter-free 60 FPS output.

3.3.2 Streaming Processor (Spark Stream)

Following data ingestion, the core processing workload is handled by Apache Spark. We evolved the deployment architecture from a monolithic prototype to a distributed cluster to ensure production scalability.

Initially, the system utilized a **Single-Node (Local Mode)** architecture where the Driver and Executor shared a single JVM (`local[*]`). While efficient for prototyping due to zero network latency, it lacked fault tolerance. To support high-scale production, we transitioned to a **Spark Standalone Cluster** on Kubernetes:

- **Spark Master (1 Replica):** Acts as the resource manager.
- **Spark Workers (2 Replicas):** Independent Pods that execute tasks in parallel, allowing for horizontal scaling.

Contrary to the theoretical expectation of linear scaling, migrating to a distributed architecture resulted in performance degradation during low-volume tests. Empirical data collected via a custom **StreamingQueryListener** revealed a trade-off between scalability and latency for small batches.

Bảng 1: Performance Comparison: Local vs. Distributed Cluster (Small Batch)

Metric	1 Worker (Baseline)	2 Workers (Distributed)	Impact
Total Runtime	126.91s	197.52s	+55.6% Slower
Throughput (Avg)	72.52 rows/sec	46.60 rows/sec	-35.7% Decrease
Trigger Execution	21,226ms	45,229ms	+113% Increase
Add Batch Time	20,806ms	42,683ms	+105% Increase

The telemetry data highlights two primary bottlenecks introduced by distribution in a low-data scenario:

1. **Scheduling & Network Latency:** The *Trigger Execution* time doubled (+113%). In a distributed setup, the Driver spends significant time serializing tasks and waiting for network responses from remote Workers. For small payloads, the Network Round-Trip Time (RTT) dominates the actual computation time.
2. **I/O Blocking:** The *Add Batch Time* (+105%) reflects the overhead of managing concurrent connections to MongoDB from multiple nodes, leading to resource contention and locking latency.

Despite the overhead in low-load scenarios, we retained the Distributed Architecture. It provides critical **Fault Tolerance** (the Master reschedules tasks if a Worker Pod fails) and **Elastic Scalability**, which are impossible in Local Mode.

3.3.3 Batch Processing (Spark Batch Model Training)

While the streaming layer prioritizes low latency for real-time ingestion, the Model Training module operates as a high-throughput, compute-intensive Batch Layer. The objective is to retrain the YOLO object detection model periodically using accumulated data to improve accuracy over time.

Unlike the distributed processing in the Speed Layer where tasks are sharded across workers, Deep Learning training using Ultralytics YOLO is inherently a monolithic process (utilizing Single-Node Multi-GPU paradigm). To integrate this into our data platform, we adopted a ****Spark-Wrapped "Fat Driver" Architecture**** deployed as a Kubernetes Job.

Spark-Wrapped "Fat Driver" Architecture In a standard Spark deployment, the Driver manages task scheduling while Executors perform the actual computation. However, our training logic defined in `model_trainer.py` executes strictly within the Python process initiating the YOLO class, without invoking distributed RDD actions.

Consequently, we inverted the standard resource allocation strategy for this specific workload:

- **Fat Driver (GPU-Enabled):** We allocate the GPU resources (e.g., NVIDIA T4) and the majority of memory (16GB+) exclusively to the Spark Driver Pod. The Driver runs the `YOLOModelTrainer.train()` method directly.
- **Zero-Executor Configuration:** We configure the Spark Session with minimal executors, effectively using Spark solely as a cluster resource manager and logging wrapper rather than a distributed compute engine.

This hybrid approach allows the application to detect and utilize the GPU attached to the Spark Driver container, as implemented in the device detection logic:

```

1 # From: model_trainer.py
2 def _detect_device(self) -> str:
3     try:
4         import torch
5         if torch.cuda.is_available():
6             return "cuda"
7         else:
8             return "cpu"
9     except Exception:
10        return "cpu"

```

Listing 10: Hybrid Device Detection Strategy

Stateless Compute & Persistent Storage Strategy Since Kubernetes Pods are ephemeral, data persistence is critical for long-running training jobs. We implemented a **Decoupled Storage Pattern** using MinIO as the persistent object storage layer.

The training pipeline operates on a strict "Check-Download-Train-Upload" lifecycle, ensuring that the compute layer remains stateless:

1. **Initialization:** The `MinIOUploader` pulls the `dataset.yaml` and raw images from MinIO to the Pod's local ephemeral storage (mounted as an `emptyDir` volume for high I/O throughput).
2. **Execution:** The YOLO engine performs training, writing temporary checkpoints to the local disk.
3. **Synchronization:** Upon completion, critical artifacts (best weights, logs, configuration) are pushed back to the object storage.

Fault Tolerance: Resumable Training Pipeline Batch training jobs can span several hours and are susceptible to preemption or failure. To ensure reliability without restarting from epoch 0, we implemented a **Resumable Training Logic**. Before initializing the model, the system queries MinIO for existing checkpoints from previous failed runs.

```

1 # From: util.py
2 def get_checkpoint_path(uploader, output_dir, checkpoint_prefix):
3     checkpoint_object_pt = f"{checkpoint_prefix}/last.pt"
4
5     if uploader.checkpoint_exists(checkpoint_object_pt):
6         print(f"Found existing checkpoint: {checkpoint_object_pt}")
7         if uploader.download_checkpoint(..., checkpoint_object_pt):
8             return str(checkpoint_object_pt)
9
10    return None

```

Listing 11: Checkpoint Recovery Mechanism

This mechanism ensures **Idempotency**: triggering the training job multiple times will either complete the remaining epochs or verify that the optimal weights have already been generated, optimizing GPU cost in the production environment.

Metrics Observability To integrate with the MLOps pipeline, the system does not rely solely on unstructured console logs. The `YOLOModelTrainer` parses the raw `results.csv` generated by Ultralytics and serializes the final performance indicators (mAP, Precision, Recall) into a structured JSON format.

By uploading this `metrics.json` to MinIO alongside the model weights, we decouple the training process from the evaluation logic, allowing downstream CD pipelines to automatically assess model quality before promoting it to production.

3.3.4 Batch Processing (Spark Batch Metric Computing)

Complementing the real-time Speed Layer, the Batch Layer performs deep retrospective analysis on accumulated traffic data. Its primary objective is to execute the YOLO object detection model on historical frames to derive high-precision metrics (density, average speed) and generate heatmaps.

Initially, this workload operated on a single Pod using Spark's `local[2]` mode. While sufficient for functional testing, this monolithic architecture suffered from resource contention and lacked the ability to scale horizontally. To address these bottlenecks, we transitioned to a fully distributed `**Spark Standalone Cluster**` on Kubernetes.

Distributed Cluster Architecture We decoupled the compute resources into a Master-Worker topology to enable parallel processing across multiple nodes.

- **Spark Master Service:** Acts as the cluster resource manager, listening on port 7077 for job submissions.
- **Spark Workers:** Replicated Pods that instantiate Executors. We utilize a unified Docker image (`quaquducc/spark-batch-computing:v1`) across the Driver, Master, and Workers to ensure binary compatibility for Python dependencies (Ultralytics, OpenCV).

Environment Standardization & Model Pre-fetching A critical challenge in distributed Deep Learning is ensuring that heavy model weights and environment configurations available on the Driver are accessible to remote Executors. We implemented a two-fold solution to prevent "Missing Dependency" failures:

1. **Context Propagation:** We explicitly propagate essential configuration keys (MinIO credentials, Image Size settings) from the Driver to all Executors via `spark.executorEnv.*`.
2. **Model Pre-fetching via InitContainers:** Since standard Spark distribution mechanisms are inefficient for large binary files, we utilized a Kubernetes `initContainer` in the Worker Pods. This container pre-downloads the YOLO weights from MinIO to a shared volume (`/tmp`) before the Spark Worker process initiates.

```
1 # From: 03-spark-batch-cluster.yaml
2 initContainers:
3   - name: init-download-model
4     command: ["/bin/sh", "-lc"]
5     args:
6       - >
7         python3 -c "from_minio import Minio; ...
8         c.get_object(..., '/tmp/yolov8_traffic_best.pt')"
9     volumeMounts:
10       - mountPath: /tmp
11         name: temp-data
```

Listing 12: Worker InitContainer for Model Synchronization

Optimized CPU Inference Strategy Running Object Detection on CPU is compute-intensive. To maximize throughput without GPUs, we optimized the execution logic using ****Pandas UDFs (Scalar Iterator)**** and input resizing.

Instead of initializing the YOLO model for every single record (which incurs massive overhead), we group data by **camera_id** and process batches using **applyInPandas**. This allows the worker to load the model once per partition and perform inference on a batch of frames efficiently.

```

1 # From: util.py
2 def run_inference_worker(key, pdf: pd.DataFrame) -> pd.DataFrame:
3     model = YOLO("/tmp/yolov8_traffic_best.pt")
4
5     IMG_SIZE = int(os.environ.get("IMG_SIZE", 320))
6
7     for index, row in tqdm(pdf.iterrows(), ...):
8         results = model.track(..., imgsz=IMG_SIZE)
9         # ... logic to calculate overlap/speed ...
10
11     return pd.DataFrame(output_rows)

```

Listing 13: Batch Inference Logic with Pandas UDF

Additionally, to mitigate I/O bottlenecks during density calculation, we utilize Spark Broadcast Variables to distribute the lightweight Road Geometry JSONs to all nodes, eliminating repetitive MinIO lookups.

Performance Outcome The transition to a distributed setup provided tangible stability improvements. In our validation run, the pipeline successfully processed approximately **59,000 detections**, generating 1-minute statistics and heatmaps without memory overflows.

Bảng 2: Batch Processing Optimization Summary

Challenge	Optimization Strategy	Impact
Model Availability	Kubernetes <code>initContainer</code>	Guaranteed model presence on Workers
CPU Latency	Resize <code>IMG_SIZE</code> 640 → 320	2x Inference Throughput
Slow I/O	Broadcast Variables	Reduced network calls for config data
Write Bottleneck	<code>insert_many</code> (MongoDB)	Optimized database ingestion

4 Lessons Learned

4.1 Lessons on Data Ingestion: Solving the "Small Files Problem" with LMDB

4.1.1 Problem Description

- **Context:** The project ingests a dataset of 10,000 traffic images.
- **Challenges:** Reading thousands of small image files (KB size) individually caused excessive I/O overhead due to OS system calls (open/close), bottlenecking the producer throughput.
- **System impact:** The ingestion layer could not sustain the high FPS required to stress-test the streaming engine.

4.1.2 Approaches Tried

- **Approach 1: Raw File Iteration.** Reading ‘.jpg’ files directly from disk. *Trade-off:* High latency due to disk seek time.
- **Approach 2: Binary Packing (LMDB).** Consolidating images into a Key-Value Lightning Memory-Mapped Database.

4.1.3 Final Solution

Data Serialization into LMDB.

- **Solution:** Pre-processed images into an LMDB file using MsgPack serialization.
- **Metric:** Ingestion throughput increased by $\approx 15\times$ compared to raw files.

4.1.4 Key Takeaways

- **Best Practice:** Avoid raw file I/O for massive small datasets. Use container formats (SequenceFile, Avro, LMDB) to leverage sequential reads and memory mapping.

4.2 Lessons on Data Processing with Spark: The "Distributed Overhead" Paradox

4.2.1 Problem Description

- **Context:** The system was initially developed in "Local Mode" (single JVM) and later migrated to a "Spark Standalone Cluster" with 2 Worker nodes to simulate a production Big Data environment.
- **Assumption:** We assumed that doubling the number of workers (Scale Out) would linearly improve performance.
- **Observation:** Contrarily, the 2-Worker cluster performed **35% slower** than the single node. Total Batch Duration increased from 126s to 197s.

4.2.2 Approaches Tried

- **Approach 1: Horizontal Scaling (2 Workers).**
 - *Metric:* Trigger Execution time jumped by +113% (21.2s $\rightarrow$ 45.2s).
 - *Root Cause Analysis:* For small micro-batches ($\approx 1,000$ rows), the overhead of network serialization, task scheduling, and database connection contention exceeded the actual compute time.
- **Approach 2: Vertical Scaling (Single Worker).**
 - *Result:* Higher throughput (72 rows/sec vs 46 rows/sec). Zero network latency between Driver and Executor.

4.2.3 Final Solution

Right-Sizing the Architecture.

- **Solution:** We acknowledged that "Scaling Out" incurs a mandatory "Coordination Tax". For the current dataset scale ($\approx 10,000$ records), a vertically scaled Single-Node architecture is mathematically optimal.
- **Architectural Decision:** Maintained the Distributed Architecture code-base for future proofing but deployed with resource-optimized configurations, accepting the overhead as an investment for future scalability (100+ cameras).

4.2.4 Key Takeaways

- **Technical Insight:** Distributed systems are not magic. Unless the data volume justifies the cost of network I/O and coordination, a simple monolithic (or local) setup will always be faster.

4.3 Lessons on Stream Processing: Handling Late Data with Watermarking

4.3.1 Problem Description

- **Context:** Traffic density is aggregated over 1-minute sliding windows.
- **Challenges:** Network jitter causes frames to arrive out of order.
- **System impact:** Aggregations based on processing time were scientifically inaccurate.

4.3.2 Approaches Tried

- **Approach 1: Processing Time.** Aggregating based on server clock. *Trade-off:* Inaccurate data representation.
- **Approach 2: Event Time.** Aggregating based on frame timestamps.

4.3.3 Final Solution

Event-Time Windowing with Watermarking.

- **Solution:** Applied `‘.withWatermark("event_time", "30 seconds")’`.
- **Result:** The system correctly handles data arriving up to 30 seconds late, updating the corresponding window state, while discarding too-old data to bound memory usage.

4.3.4 Key Takeaways

- **Recommendation:** Always decouple Event Time from Processing Time in real-time analytics.

4.4 Lessons on Data Storage: Hybrid Storage Strategy (Hot vs. Cold)

4.4.1 Problem Description

- **Context:** The system generates both raw image data (high volume) and analytical insights (low volume, high value).
- **Challenges:** Storing raw images in a database is expensive and slow; storing insights in a file system makes querying difficult.

4.4.2 Approaches Tried

- **Approach 1: Single Store.** Storing everything in MongoDB. *Trade-off:* Database bloat and slow backups.
- **Approach 2: Hybrid Store.** Separating storage concerns.

4.4.3 Final Solution

Polyglot Persistence.

- **Solution:**
 - **Cold Data (MinIO):** Raw images and configuration masks are stored in Object Storage.
 - **Hot Data (MongoDB):** Aggregated metrics are stored in NoSQL for low-latency querying.

4.4.4 Key Takeaways

- **Best Practice:** Align the storage engine with the data access pattern (Read-heavy analytics vs. Write-heavy archiving).

4.5 Lessons on System Integration: Service Discovery in Kubernetes

4.5.1 Problem Description

- **Context:** Components (Spark, Kafka, Mongo) run in dynamic pods with ephemeral IPs.
- **Challenges:** Hardcoding IP addresses causes immediate failure upon pod restarts.

4.5.2 Approaches Tried

- **Approach 1: Hardcoded IPs.** *Trade-off:* Extremely fragile.
- **Approach 2: NodePort.** *Trade-off:* Limited portability.

4.5.3 Final Solution

K8s Internal DNS Resolution.

- **Solution:** Utilized Kubernetes Services ('ClusterIP' and Headless Services). Components reference each other via DNS names (e.g., 'my-kafka:9092').
- **Result:** Zero-config connectivity during scaling or restarting.

4.5.4 Key Takeaways

- **Recommendation:** Never rely on Pod IPs. Use Service abstraction for robust integration.

4.6 Lessons on Performance Optimization: Decoupling Training Workloads for GPU Efficiency

4.6.1 Problem Description

- **Context:** The pipeline trains object detection models (YOLOv8) using high-volume traffic camera data via Spark and Kafka.
- **Challenges:** End-to-end training was slow due to CPU-bound processing within Spark, leading to memory pressure during image decoding and tight coupling between data processing and training.
- **System impact:** Long training cycles and limited throughput made system tuning and debugging difficult.

4.6.2 Approaches Tried

- **Approach 1: Monolithic Spark-based training.** Spark handled Kafka consumption, preprocessing, and training in one job. *Trade-off:* Simple but inefficient; hard to scale as training ran on CPU.
- **Approach 2: Decoupled GPU Worker.** Separating orchestration from execution.

4.6.3 Final Solution

Decoupled Orchestration and Execution.

- **Solution:** Implemented a Spark Dispatcher for orchestration and dedicated GPU Workers for execution. Spark runs in 'local[*]' mode for lightweight coordination, while GPU workers auto-detect CUDA and use AMP (Automatic Mixed Precision).
- **Implementation:** Increased batch size ($1 \rightarrow 16$) and enabled shared memory ('/dev/shm') to prevent OOM.
- **Result:** Faster training time per epoch, improved GPU utilization, and reduced memory-related failures.

4.6.4 Key Takeaways

- **Technical Insight:** Decoupling compute-heavy workloads improves performance. GPUs should not be embedded directly inside orchestration jobs.

4.7 Lessons on Monitoring & Debugging: Debugging Distributed UDF Failures

4.7.1 Problem Description

- **Context:** Errors occurring inside Python UDFs on remote workers are often swallowed or appear as generic Java exceptions in the Driver.
- **Challenges:** Difficulty in performing Root Cause Analysis (RCA) for specific image processing failures.

4.7.2 Approaches Tried

- **Approach 1: Driver Logs.** *Trade-off:* Lack of context.
- **Approach 2: Exception Wrapping.**

4.7.3 Final Solution

Exception Wrapping and Payload Logging.

- **Solution:** Wrapped UDF logic in ‘try-except’ blocks. Instead of crashing, errors are returned as JSON messages: “status”: “failed”, “error”: “...”.
- **Result:** Errors are preserved in the data stream and stored in MongoDB, allowing post-mortem analysis via queries.

4.7.4 Key Takeaways

- **Best Practice:** Distributed systems should fail gracefully. Treat errors as data.

4.8 Lessons on Scaling: Event-Driven Horizontal Scaling Strategies

4.8.1 Problem Description

- **Context:** The system must scale with increasing camera count and data volume while remaining cost-efficient.
- **Challenges:** Single Spark training jobs had limited scalability, leading to resource contention between Spark orchestration and the training process.
- **System impact:** Poor horizontal scalability and underutilized GPUs.

4.8.2 Approaches Tried

- **Approach 1: Vertical Scaling (Old Pipeline).** Increasing CPU/memory for the Spark job. *Trade-off:* Easy to implement but expensive and capped by node capacity.
- **Approach 2: Horizontal Scaling via Kafka.** Distributing load using partitions.

4.8.3 Final Solution

Kafka-Driven Horizontal Scaling.

- **Solution:** Used Kafka as a scaling backbone with camera-based partitioning. Producers, dispatchers, and workers scale independently.
- **Implementation:** GPU workers are isolated from the Kubernetes cluster management logic and use dynamic Kafka consumer groups.
- **Result:** Ability to add more GPU workers without pipeline changes and improved cost-to-performance ratio.

4.8.4 Key Takeaways

- **Best Practice:** Use message queues to decouple scaling domains. Event-driven design with stateless workers simplifies horizontal scaling.

4.9 Lessons on Data Quality & Testing: Schema Enforcement in Streaming

4.9.1 Problem Description

- **Context:** JSON data from producers can sometimes be malformed or missing fields.
- **Challenges:** ‘SchemaMismatch’ errors can terminate the entire streaming job.

4.9.2 Approaches Tried

- **Approach 1: Schema Inference.** *Trade-off:* Fragile in production.
- **Approach 2: Explicit Schema.**

4.9.3 Final Solution

Strict Schema Definition with Error Handling.

- **Solution:** Defined explicit ‘StructType’ schema. Used Spark’s default ‘PERMISSIVE’ mode to parse malformed records as ‘null’, then filtered them out.
- **Result:** The pipeline is resilient to bad data injection.

4.9.4 Key Takeaways

- **Best Practice:** Never use schema inference in production streaming. Enforce contracts explicitly.

4.10 Lessons on Security & Governance: Secret Management Trade-offs

4.10.1 Problem Description

- **Context:** Managing credentials for MinIO and MongoDB.
- **Challenges:** Balancing development speed vs. security compliance.

4.10.2 Approaches Tried

- **Approach 1: Env Vars.** Hardcoding passwords in YAML. *Trade-off:* Insecure, visible in Git.
- **Approach 2: K8s Secrets.** Mounting secrets as volumes.

4.10.3 Final Solution

Environment Variables (Academic Scope).

- **Solution:** Used Environment Variables for ease of debugging in this academic project.
- **Compliance Note:** We acknowledge this violates security best practices. In production, ‘Kubernetes Secrets’ and RBAC would be mandatory.

4.10.4 Key Takeaways

- **Insight:** Security is a continuous trade-off. Credentials should never be committed to Version Control.

4.11 Lessons on Fault Tolerance: Ensuring Exactly-Once Semantics with Checkpointing

4.11.1 Problem Description

- **Context:** Long-running streaming jobs are prone to crashes.
- **Challenges:** Losing the "offset" position in Kafka means either data loss or data duplication upon restart.

4.11.2 Approaches Tried

- **Approach 1: No Checkpointing.** *Trade-off:* Data inconsistency.
- **Approach 2: Persistent Checkpointing.**

4.11.3 Final Solution

Checkpointing to Persistent Storage.

- **Solution:** Configured ‘checkpointLocation’ to a Persistent Volume Claim (PVC).
- **Result:** Upon restart, Spark reads the checkpoint, restores the aggregation state, and resumes from the exact last processed offset.

4.11.4 Key Takeaways

- **Best Practice:** Checkpointing is the backbone of fault tolerance in Structured Streaming.

5 Conclusion and Future Work

5.1 Conclusion

This project has successfully demonstrated the implementation of a robust, Kappa-style data pipeline tailored for real-time traffic image analysis. By leveraging modern Big Data technologies, we have built a system capable of handling high-throughput data flows—from ingestion via Apache Kafka to distributed stream processing with PySpark and real-time visualization through Metabase.

The architecture proves that integrating Computer Vision workloads within a Kubernetes-orchestrated environment is not only feasible but also highly scalable. Our implementation of fixed partitioning and in-memory caching for road geometries highlights the potential for optimizing low-latency pipelines in resource-constrained environments. Ultimately, this system provides a foundation for smart city infrastructure, offering actionable insights into traffic density and vehicle behavior.

5.2 Future Work

While the current pipeline meets the core objectives, several avenues for enhancement remain:

- **Live Stream Integration:** Transition from processing static LMDB datasets to consuming live video feeds directly from IP cameras.
- **GPU Acceleration:** Implement Triton Inference Server or Spark-GPU plugins to accelerate the OpenCV and Deep Learning UDFs.
- **Advanced Security:** Implement mTLS (mutual TLS) for communication between Kubernetes pods and fine-grained Role-Based Access Control (RBAC) for data access.

Acknowledgments

We would like to express our deepest gratitude to our instructor, **Dr. Tran Viet Trung**, for his invaluable guidance, insightful feedback, and constant encouragement throughout this project. We also extend our thanks to the vibrant open-source communities behind **Apache Spark**, **Apache Kafka**, and **Kubernetes**. Their comprehensive documentation and robust tools made it possible for us to bridge the gap between theoretical concepts and practical implementation.

Work Distribution

Member	Work Load
Lại Trí Dũng 20225486	Spark Stream + Redis + Metabase
Nguyễn Trọng Tâm 20225527	Producer + Spark Stream Optimization
Vũ Hữu An 20225497	Spark Batch Model Training + (GPU Optimization)
Lưu Thiện Việt Cường 20225477	Spark Batch Metric Computing
Đàm Quang Đức 20225483	Spark Batch Metric Computing Optimization